

Laboratorio di Programmazione

Esercizi di esame — livello difficile (05, 06, 07)

Esercizio 05 — Regressione lineare su CSV

Un file CSV contiene due colonne numeriche **x** e **y**. Crea la classe `CSVDataFile`, inizializzata col nome del file, con un metodo `get_data()` che restituisce la lista delle coppie lette. Crea poi la classe `LinearRegression`, inizializzata con i dati, con il metodo `fit()` che calcola i coefficienti della retta di regressione (**m** e **q**) e `predict(x)` che stima il valore di **y** per un dato **x**.

ESEMPIO D'USO

```
data_file = CSVDataFile(name='data.csv')
data = data_file.get_data()
model = LinearRegression(data)
model.fit()
print(model.predict(5.0)) # es. 23.7
print(model.get_r_squared()) # es. 0.94
```

SPECIFICHE

Classe `CSVDataFile`

- Il file viene aperto solo alla chiamata di `get_data()`; se non esiste o non è leggibile, alzare `ExamException`.
- Righe con valori non numerici o mancanti vanno ignorate stampando un messaggio; tutto il resto deve procedere normalmente.
- Se il file non contiene almeno 2 righe valide, alzare `ExamException`.
- La prima riga è l'intestazione (x,y) e va saltata.

Classe `LinearRegression`

- Calcolare **m** e **q** con le formule dei minimi quadrati ordinari (OLS), senza usare librerie esterne.
- Se `predict()` viene chiamato prima di `fit()`, alzare `ExamException`.
- Se tutti i valori di **x** sono uguali, la regressione non è definita: alzare `ExamException` in `fit()`.
- Aggiungere il metodo `get_r_squared()` che ritorna il coefficiente R^2 (valore tra 0 e 1).
- Se R^2 non può essere calcolato (varianza di **y** nulla), alzare `ExamException`.

Gestione input e casi limite

- Se i dati passati a `LinearRegression` non sono una lista di coppie, alzare `ExamException`.
- `predict()` deve accettare sia interi che float; qualsiasi altro tipo deve alzare `ExamException`.

Esercizio 06 — Interpolazione e ricampionamento di time series

Un file CSV contiene una time series con timestamp giornalieri nel formato YYYY-MM-DD e valori numerici, ma con possibili buchi (giorni mancanti). Crea la classe `CSVDailySeriesFile` con metodo `get_data()` che restituisce la lista dei dati letti. Crea poi la classe `TimeSeriesResampler`, inizializzata con i dati, con due metodi: `interpolate()`, che riempie i giorni mancanti con interpolazione lineare, e `resample(frequency)`, che ricampiona la serie alla frequenza indicata ("weekly" o "monthly") calcolando la media dei valori nel periodo.

ESEMPIO D'USO

```
f = CSVDailySeriesFile(name='daily.csv')
data = f.get_data()

resampler = TimeSeriesResampler(data)
filled = resampler.interpolate()
weekly = resampler.resample("weekly")
# {"2024-W01": 112.4, "2024-W02": 118.7, ...}
monthly = resampler.resample("monthly")
# {"2024-01": 115.2, "2024-02": 121.0, ...}
```

SPECIFICHE

Classe `CSVDailySeriesFile`

- Il file viene aperto solo in `get_data()`; se non esiste o non è leggibile, alzare `ExamException`.
- Righe con data non in formato YYYY-MM-DD, valore non numerico, negativo o vuoto vanno ignorate (stampare un messaggio).
- Timestamp fuori ordine o duplicati: alzare `ExamException`.
- Se il file non contiene almeno due righe valide, alzare `ExamException`.

Metodo `interpolate()`

- Restituisce una nuova lista con tutti i giorni dell'intervallo coperti; i giorni mancanti hanno il valore interpolato linearmente tra il valore precedente e successivo disponibile.
- Non modificare la lista originale: restituire sempre una nuova struttura dati.
- Non interpolare oltre il primo o l'ultimo dato disponibile (nessuna estrapolazione ai bordi).
- I valori interpolati devono essere arrotondati a due decimali.

Metodo `resample(frequency)`

- Se `frequency` non è "weekly" o "monthly", alzare `ExamException`.
- Il metodo usa i dati originali (non interpolati) salvo diversa scelta di progetto, purché documentata.
- Periodi con un solo valore disponibile: usare quel valore come media.
- Le chiavi del dizionario di output usano il formato ISO: YYYY-Www per le settimane, YYYY-MM per i mesi.
- Non usare librerie esterne come pandas; la gestione delle date va fatta con il modulo standard `datetime`.

Esercizio 07 — Analisi di un log di eventi da file

Un file di testo contiene un log di eventi nel formato `YYYY-MM-DD HH:MM:SS | LIVELLO | messaggio` (es. `2024-03-01 14:32:07 | ERROR | connessione persa`). Crea la classe `LogFile` con metodo `get_events()` che restituisce la lista degli eventi come dizionari. Crea poi la classe `LogAnalyzer`, inizializzata con la lista di eventi, con tre metodi: `filter_by_level(level)`, che restituisce solo gli eventi del livello indicato; `compute_hourly_distribution()`, che ritorna un dizionario con il conteggio degli eventi per ora del giorno; e `find_bursts(window_minutes, threshold)`, che individua le finestre temporali in cui si verificano almeno **threshold** eventi.

ESEMPIO D'USO

```
log = LogFile(name='app.log')
events = log.get_events()
# [{"timestamp": "2024-03-01 14:32:07",
#  "level": "ERROR", "message": "connessione persa"}, ...]

analyzer = LogAnalyzer(events)
errors = analyzer.filter_by_level("ERROR")
dist = analyzer.compute_hourly_distribution()
# {0: 3, 1: 0, ..., 23: 12}
bursts = analyzer.find_bursts(window_minutes=5, threshold=10)
# [{"start": "2024-03-01 14:30:00",
#  "end": "2024-03-01 14:35:00", "count": 13}, ...]
```

SPECIFICHE

Classe `LogFile`

- Il file viene aperto solo in `get_events()`; se non esiste o non è leggibile, alzare `ExamException`.
- Righe non conformi al formato (campi mancanti, timestamp non parsabile) vanno ignorate stampando un messaggio.
- Il livello viene convertito in uppercase; il messaggio viene trimmed degli spazi.
- Timestamp fuori ordine: alzare `ExamException`. Timestamp duplicati: ignorare la riga e stampare un messaggio (non alzare eccezione).
- Se il file non contiene nessuna riga valida, alzare `ExamException`.

Metodo `filter_by_level(level)`

- Se `level` non è una stringa, alzare `ExamException`.
- Se nessun evento corrisponde al livello, restituire una lista vuota (non alzare eccezione).
- Il confronto sul livello deve essere case-insensitive.

Metodo `compute_hourly_distribution()`

- Restituisce sempre un dizionario con tutte le 24 ore (0–23) come chiavi intere, anche se il conteggio è zero.
- Se la lista di eventi è vuota, alzare `ExamException`.

Metodo `find_bursts(window_minutes, threshold)`

- La finestra è scorrevole: si sposta di un minuto alla volta sull'intero arco temporale del log.
- `window_minutes` deve essere un intero positivo e `threshold` un intero maggiore di 1; altrimenti alzare `ExamException`.
- Burst sovrapposti (finestre consecutive che si includono a vicenda) vanno tutti restituiti separatamente.
- Se non viene trovato nessun burst, restituire una lista vuota.
- Non usare librerie esterne; usare solo il modulo standard `datetime`.

In tutti gli esercizi, le eccezioni vanno alzate come istanze di `ExamException(Exception)`. Chiamate il file `esame_matricola.py`. Commentate il codice.